EECE 7205: Fundamentals of Computer Engineering

Fall 2026

In-Class Work, Part 2: Binary Search Trees

Topic: Building, searching and deleting in a binary search tree (rules I1–I4, S1–S4 and D1–D3)

Student Name	Catalina Buitano
NUID	N/A
Email	buitano.c@northeastern.edu
Instructor	Dr. Fatemah Nafa
Due Date	October 23rd, 2026

Problem 1: Building the Tree (Insert)

Case 3.

Insert a larger key (insert 70)

(a) Rule(s) and Explanation

Every comparison in order:

Step	Check	Result	Rule
1	70 vs 50	$70 > 50 \rightarrow$ go right	I3
2	50.right	NULL $\rightarrow$ empty spot, attach 70	I1
	Total	1 comparison	

Rules I3, then I1. Compare 70 with the root: $70 > 50$, so by the BST property 70 belongs in the **right** subtree of 50 (I3). The right child of 50 is NULL, so this is an empty spot and 70 is attached there as a **leaf** (I1). One comparison was needed. This is the mirror image of Case 2.

(b) Pseudocode

Algorithm 1 INSERT (Case 3 fragment: larger key, empty right spot)

Input: key = 70, and the tree {50, 30}

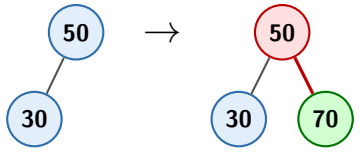
Output: the tree with 70 attached as the right child of 50

Initial call: INSERT(70)

```
1. cur ← root                                   ▷ start at the root (50)
2. if key > cur.key then                        ▷ I3: go right
3.     if cur.right = NULL then                 ▷ I1: empty spot
4.         cur.right ← newNode
5.         return
6.     end if
7.     cur ← cur.right                          ▷ otherwise keep walking down
8. end if
```

Explanation: Line 2 applies I3: $70 > 50$, so 70 must go in 50's right subtree. Line 3 finds that the right subtree is empty, so line 4 attaches 70 there as a leaf. No other node moves, so the BST property still holds.

(c) Tree Before and After



Before: {50, 30}. After: 70 is the right child of 50.

Case 4.
Insert deep in the tree, turning left and right (insert 20, 40, 60, 80, then 35)

(a) Rule(s) and Explanation

First the four setup inserts, each worked out:

Insert	Path (comparisons)	Rules	Placed as
20	$20 < 50$ left, $20 < 30$ left	I2, I2, I1	left child of 30
40	$40 < 50$ left, $40 > 30$ right	I2, I3, I1	right child of 30
60	$60 > 50$ right, $60 < 70$ left	I3, I2, I1	left child of 70
80	$80 > 50$ right, $80 > 70$ right	I3, I3, I1	right child of 70

Now insert 35, every comparison in order:

Step	Check	Result	Rule	What it tells us
1	35 vs 50	$35 < 50 \rightarrow$ left to 30	I2	35 is smaller than the root
2	35 vs 30	$35 > 30 \rightarrow$ right to 40	I3	35 is larger than 30
3	35 vs 40	$35 < 40 \rightarrow$ go left	I2	35 is smaller than 40
4	40.left	NULL $\rightarrow$ attach 35	I1	empty spot found
	Total	3 comparisons (50, 30, 40)		

Rules I2, I3, I2, then I1. Each comparison is local to one node: it only asks whether 35 is smaller or larger than *that* node's key. 35 is smaller than 50 and smaller than 40, so it goes **left** at those two nodes. It is larger than 30, so it goes **right** at 30. The path left, right, left records exactly where 35 sits in sorted order: $30 < 35 < 40 < 50$.

(b) Pseudocode

Algorithm 2 INSERT (Case 4 fragment: the loop that walks down)

Input: key = 35, and the tree {50, 30, 70, 20, 40, 60, 80}

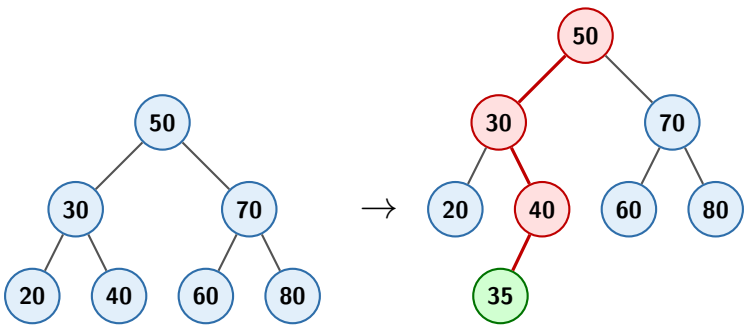
Output: the tree with 35 attached as the left child of 40

Initial call: INSERT(35)

```
1.  cur ← root
2.  while true do
3.      if key < cur.key then                          ▷ I2: go left
4.          if cur.left = NULL then                    ▷ I1: empty spot
5.              cur.left ← newNode
6.              return
7.          end if
8.          cur ← cur.left
9.      else                                           ▷ I3: go right
10.         if cur.right = NULL then                   ▷ I1: empty spot
11.             cur.right ← newNode
12.             return
13.         end if
14.         cur ← cur.right
15.     end if
16. end while
```

Explanation: Every pass of the loop makes one comparison and moves *cur* down one level, so the loop always ends at an empty spot (lines 5 and 11). Each turn keeps the key in the only subtree where the BST property allows it, so the empty spot it reaches is the correct one.

(c) Tree Before and After Inserting 35



Before: 7 keys. After: 35 is the left child of 40 (path 50 → 30 → 40 shown in red).

Case 5.
Insert a key that is already in the tree (insert 40 again)

(a) Rule(s) and Explanation

Finishing the tree:

Insert	Path (comparisons)	Rules	Placed as
45	$45 < 50$ left, $45 > 30$ right, $45 > 40$ right	I2, I3, I3, I1	right child of 40
65	$65 > 50$ right, $65 < 70$ left, $65 > 60$ right	I3, I2, I3, I1	right child of 60
75	$75 > 50$ right, $75 > 70$ right, $75 < 80$ left	I3, I3, I2, I1	left child of 80

Now insert 40 again:

Step	Check	Result	Rule
1	40 vs 50	$40 < 50 \rightarrow$ left to 30	I2
2	40 vs 30	$40 > 30 \rightarrow$ right to 40	I3
3	40 vs 40	$40 = 40 \rightarrow$ duplicate, stop	I4
	Total	3 comparisons, tree unchanged	

Rules I2, I3, then I4. The walk reaches the existing 40 and the key is equal, so nothing is inserted and the tree does not change. A BST must not store 40 twice because the BST property uses strict inequalities: a second 40 would have to be *smaller* than 40 to go into the left subtree or *larger* than 40 to go into the right subtree, and it is neither. Also, a search for 40 always stops at the first 40 it meets, so a second copy could never be found, and deleting 40 would be ambiguous. The BST works as a set of unique keys.

(b) Pseudocode

Algorithm 3 INSERT (Case 5 fragment: duplicate key)

Input: key = 40, and the full 11-key tree

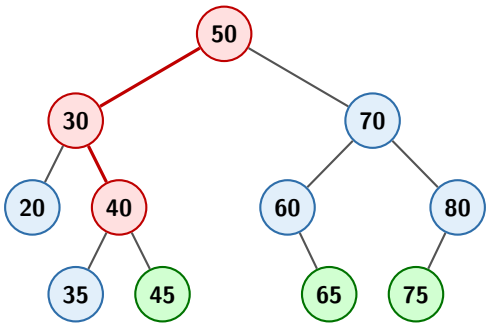
Output: the same tree, unchanged

Initial call: INSERT(40)

```
1. if key = cur.key then          ▷ I4: key already present
2.     return                     ▷ do nothing; tree unchanged
3. end if
```

Explanation: This test runs first at every node, before the left and right tests, so an equal key is caught as soon as it is reached. Returning without creating a node is what keeps every key unique.

(c) Final Tree of arr (all 11 keys)



45, 65 and 75 (green) complete the tree. Inserting 40 again (path in red) stops at the existing 40 and changes nothing.

Case 6.
Insert keys that arrive in sorted order (insert 20, 30, 40, 50, 60 into an empty tree)

(a) Rule(s) and Explanation

Insert	Compared with	Rules	Comparisons
20	none (tree empty)	I1	0
30	20	I3, I1	1
40	20, 30	I3, I3, I1	2
50	20, 30, 40	I3, I3, I3, I1	3
60	20, 30, 40, 50	I3, I3, I3, I3, I1	4

Total comparisons:

$$T = 0 + 1 + 2 + 3 + 4$$

(the i-th key is compared with every key already on the chain)

$$T = 10$$

(add the terms)

$$T(n) = \sum_{i=0}^{n-1} i = \frac{n(n-1)}{2} = \frac{5 \cdot 4}{2} = 10$$

(general formula, checked with $n = 5$)

Height:

$$h = n - 1 = 5 - 1 = 4$$

(every node except the last has exactly one child, so each key adds one edge)

Rule I3 every time (then I1 to attach). Each new key is larger than every key already in the tree, so it always goes right. The tree degenerates into a right-leaning chain, which is really a linked list. Its height is 4 for only 5 keys, while the tree of Case 5 holds 11 keys with height 3. So search, insert and delete on a chain cost $\Theta(n)$ instead of $\Theta(\log n)$. The **order** the keys are inserted in decides the shape of the tree.

(b) Pseudocode

Algorithm 4 BUILD-FROM-ARRAY

Input: data, an array of n keys

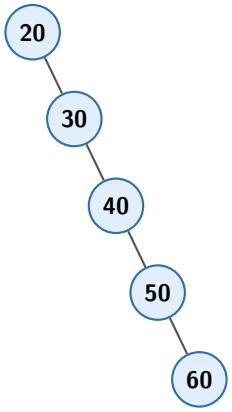
Output: the root of the BST holding every distinct key of data

Initial call: BUILD-FROM-ARRAY(data, n)

```
1. procedure BUILD-FROM-ARRAY(data, n)
2.     root ← NULL                          ▷ start with an empty tree
3.     for i ← 0 to n − 1 do                ▷ read keys in array order
4.         INSERT(data[i])                  ▷ each insert applies I1–I4
5.     end for
6.     return root
7. end procedure
```

Explanation: The loop calls INSERT once for every key, in array order. Each INSERT keeps the BST property, so after the last one the whole tree is a BST. Because the keys are taken in array order, the same keys in a different order can give a different tree, as this case shows.

(c) Tree



Sorted input gives a chain of height 4.

(g) Table for Inserting All of arr

Step	Insert	Nodes compared (in order)	Rule(s)	Parent	Left / Right
1	50	none	I1	none (root)	root
2	30	50	I2, I1	50	Left
3	70	50	I3, I1	50	Right
4	20	50, 30	I2, I2, I1	30	Left
5	40	50, 30	I2, I3, I1	30	Right
6	60	50, 70	I3, I2, I1	70	Left
7	80	50, 70	I3, I3, I1	70	Right
8	35	50, 30, 40	I2, I3, I2, I1	40	Left
9	45	50, 30, 40	I2, I3, I3, I1	40	Right
10	65	50, 70, 60	I3, I2, I3, I1	60	Right
11	75	50, 70, 80	I3, I3, I2, I1	80	Left

(h) Complete INSERT, Height and Deepest Insert

Algorithm 5 INSERT (Cases 1 to 5 combined)

Input: key, the key to insert

Output: the tree with key added, or unchanged if key was already present

Initial call: INSERT(key), once for each key

```
1.  procedure INSERT(key)
2.      if root = NULL then                          ▷ I1: empty tree (Case 1)
3.          root ← new node with key
4.          return
5.      end if
6.      cur ← root
7.      while true do
8.          if key = cur.key then                    ▷ I4: duplicate (Case 5)
9.              return
10.         else if key < cur.key then               ▷ I2: go left (Cases 2, 4)
11.             if cur.left = NULL then              ▷ I1: empty spot
12.                 cur.left ← new node with key
13.                 return
14.             end if
15.             cur ← cur.left
16.         else                                     ▷ I3: go right (Cases 3, 4)
17.             if cur.right = NULL then             ▷ I1: empty spot
18.                 cur.right ← new node with key
19.                 return
20.             end if
21.             cur ← cur.right
22.         end if
23.     end while
24. end procedure
```

Explanation: Lines 2 to 5 handle the empty tree. Inside the loop the duplicate test (line 8) comes first, then the left (line 10) and right (line 16) tests, so exactly one rule applies at each node. Every pass moves one level down, so the loop stops after at most h + 1 comparisons.

Height of the final tree and the deepest insert:

$$h = \text{edges from the root to the deepest leaf} = 3$$

(the deepest leaves 35, 45, 65 and 75 are three edges below 50)

$$\text{levels} = h + 1 = 3 + 1 = 4$$

(levels are numbered 0 to h)

$$\text{comparisons for a deepest insert} = h = 3$$

(a new leaf at depth 3 is compared once with each of the 3 nodes above it, e.g. 50, 30, 40 for 35)

So the final tree has **height 3**, and the deepest inserts (35, 45, 65, 75) need **3 comparisons** each.

Problem 2: Searching the Tree

Case 3.
The target is in the right subtree, path turns (search 65)

(a) Rule at Each Node and Explanation

Step	Node	Comparison	Rule	Move
1	50	$65 > 50$	S4	go right to 70
2	70	$65 < 70$	S3	go left to 60
3	60	$65 > 60$	S4	go right to 65
4	65	$65 = 65$	S2	FOUND
		Total		4 comparisons

Rules S4, S3, S4, then S2. The search starts at the root and makes one comparison per level. Because $65 > 50$, the whole left subtree (30 and everything below it) is skipped. At 70 the path turns left because $65 < 70$, and at 60 it turns right because $65 > 60$. The path turns because 65 lies between 60 and 70 in sorted order. 65 is found after **4 comparisons**.

(b) Pseudocode

Algorithm 6 SEARCH

Input: target = 65, and the final tree

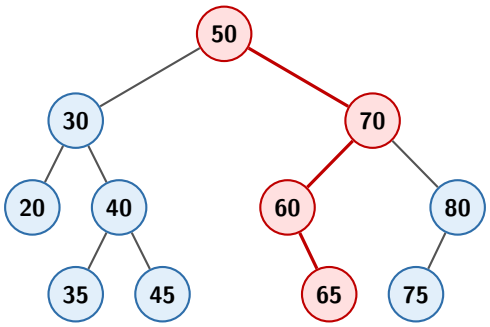
Output: FOUND

Initial call: SEARCH(65)

```
1.  procedure SEARCH(target)
2.      cur ← root
3.      while cur ≠ NULL do
4.          if target = cur.key then                ▷ S2: found
5.              return FOUND
6.          else if target < cur.key then          ▷ S3: go left
7.              cur ← cur.left
8.          else                                   ▷ S4: go right
9.              cur ← cur.right
10.         end if
11.     end while
12.     return NOT FOUND                           ▷ S1: reached NULL
13. end procedure
```

Explanation: For 65 the loop runs four times: line 9 (S4), line 7 (S3), line 9 (S4), then line 5 returns FOUND. Each move keeps the target inside the only subtree where it can be, so if 65 is in the tree the loop must reach it.

(c) Tree with the Search Path



Search path: 50 → 70 → 60 → 65. Found after 4 comparisons.

Case 4.
The target is not in the tree (search 33)

(a) Rule at Each Node and Explanation

Step	Node	Comparison	Rule	Move
1	50	$33 < 50$	S3	go left to 30
2	30	$33 > 30$	S4	go right to 40
3	40	$33 < 40$	S3	go left to 35
4	35	$33 < 35$	S3	go left: 35.left is NULL
5	NULL	none	S1	NOT FOUND
		Total		4 comparisons

Rules S3, S4, S3, S3, then S1. The search stops at the **NULL left pointer of 35**. Reaching NULL proves 33 is not in the tree: if 33 were in the tree, the BST property would force it to be exactly at this spot, and the spot is empty. This is also exactly where an insert would put it, so **if inserted, 33's parent would be 35, and 33 would be its left child.**

(b) Pseudocode

Algorithm 7 SEARCH (Case 4: leaving the loop at NULL)

Input: target = 33, and the final tree

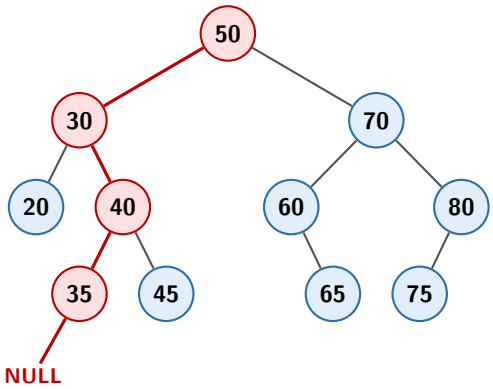
Output: NOT FOUND

Initial call: SEARCH(33)

```
1. cur ← root
2. while cur ≠ NULL do
3.     if target < cur.key then                ▷ S3: go left (50, 40, 35)
4.         cur ← cur.left
5.     else                                    ▷ S4: go right (30)
6.         cur ← cur.right
7.     end if
8. end while
9. return NOT FOUND                            ▷ S1: fell off the tree
```

Explanation: This is the same SEARCH loop with the S2 test left out, because it is never true for 33. After the fourth comparison *cur* becomes 35.left = NULL, the loop condition on line 2 fails, and line 9 returns NOT FOUND.

(c) Tree with the Search Path



Search path: 50 → 30 → 40 → 35 → NULL. Not found after 4 comparisons; 33 would be the left child of 35.

Case 5.

The target is larger than every key (search 90)

(a) Rule at Each Node and Explanation

Step	Node	Comparison	Rule	Move
1	50	$90 > 50$	S4	go right to 70
2	70	$90 > 70$	S4	go right to 80
3	80	$90 > 80$	S4	go right: 80.right is NULL
4	NULL	none	S1	NOT FOUND
		Total		3 comparisons

Rules S4, S4, S4, then S1. A target larger than every key always goes right, so the path is the rightmost spine of the tree: 50, 70, 80. It ends at the NULL right pointer of 80, the maximum key. Only 3 comparisons are needed because the right spine has only 3 nodes. If 90 were inserted, it would become the right child of 80.

(b) Pseudocode

Algorithm 8 SEARCH (Case 5: only the S4 branch runs)

Input: target = 90, and the final tree

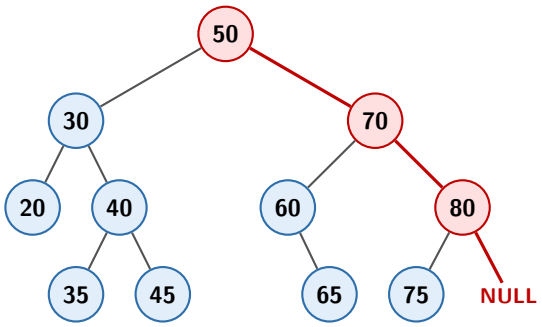
Output: NOT FOUND

Initial call: SEARCH(90)

1. $cur \leftarrow root$
2. **while** $cur \neq$ NULL **do**
3. $\quad cur \leftarrow cur.right$ ▷ S4 every time: $90 > cur.key$
4. **end while**
5. **return** NOT FOUND ▷ S1

Explanation: In the full SEARCH, the S2 and S3 tests are false at every node for 90, so only the S4 branch runs. The loop walks 50 → 70 → 80 → NULL and returns NOT FOUND.

(c) Tree with the Search Path



Search path: 50 → 70 → 80 → NULL. Not found after 3 comparisons.

Case 6.

Find the minimum and the maximum key

(a) Rule and Explanation

Step	Min: at node	Check	Max: at node	Check
1	50	left is 30 → move	50	right is 70 → move
2	30	left is 20 → move	70	right is 80 → move
3	20	left is NULL → stop	80	right is NULL → stop
	Minimum = 20		**Maximum = 80**	

Rule: follow left pointers for the minimum and right pointers for the maximum, and stop when the next pointer is NULL. By the BST property every key smaller than a node is in its left subtree, so the smallest key is the node with no left child at the end of the left spine. In

the same way, the largest key is at the end of the right spine. No key comparisons are needed (0 comparisons), only pointer checks, with 2 moves each. The minimum is not necessarily the deepest node: 20 is at depth 2, while 35 is deeper.

(b) Pseudocode

Algorithm 9 FIND-MIN and FIND-MAX

Input: node, the root of a tree or subtree

Output: the node with the smallest (or largest) key, or NULL if the tree is empty

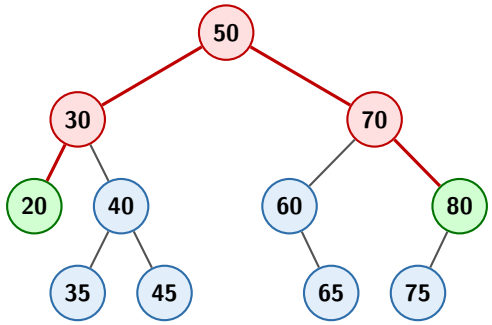
Initial call: FIND-MIN(root) and FIND-MAX(root)

```
1.  procedure FIND-MIN(node)
2.      if node = NULL then return NULL
3.      end if
4.      while node.left ≠ NULL do                    ▷ keep going left
5.          node ← node.left
6.      end while
7.      return node                                  ▷ leftmost node = minimum
8.  end procedure

9.  procedure FIND-MAX(node)
10.     if node = NULL then return NULL
11.     end if
12.     while node.right ≠ NULL do                   ▷ keep going right
13.         node ← node.right
14.     end while
15.     return node                                  ▷ rightmost node = maximum
16. end procedure
```

Explanation: Each loop stops at the first node whose next pointer is NULL. For FIND-MIN that is 20, because nothing in the tree is smaller than 20. FIND-MIN is also what DELETE uses in Problem 3 to find the inorder successor.

(c) Tree with Both Paths



Min path 50 → 30 → 20 and max path 50 → 70 → 80. The minimum 20 and maximum 80 are shown in green.

Summary Table

Case	Target	Nodes compared (in order)	Rules	Result	Comparisons
1	50	50	S2	found	1
2	45	50, 30, 40, 45	S3, S4, S4, S2	found	4
3	65	50, 70, 60, 65	S4, S3, S4, S2	found	4
4	33	50, 30, 40, 35	S3, S4, S3, S3, S1	not found (would be left child of 35)	4
5	90	50, 70, 80	S4, S4, S4, S1	not found (would be right child of 80)	3
6	min / max	50, 30, 20 / 50, 70, 80 (pointer moves only)	follow left / follow right to NULL	min 20, max 80	0

(g) Largest Number of Comparisons

$$\text{comparisons} \leq \text{nodes on the longest root-to-leaf path}$$

(each comparison happens at a different level)

$$\text{nodes on that path} = h + 1$$

(a path of h edges has h + 1 nodes)

$$\text{maximum comparisons} = 3 + 1 = 4$$

(h = 3 for this tree)

The largest number of comparisons any search can need is **4**, which is $\mathbf{h + 1}$. The worst case is a search that ends at one of the deepest nodes (35, 45, 65 or 75), or an unsuccessful search that falls off one of them, like 33. In general a search costs $\Theta(h)$: about $\log_2 n$ for a balanced tree, but up to n for a chain like Problem 1, Case 6.

Problem 3: Deleting from the Tree

Case 3.
Delete a node with only a right child (delete 60)

(a) Rule(s) and Explanation

Step	Check	Result	Rule
1	60 vs 50	$60 > 50 \rightarrow$ go right to 70	search
2	60 vs 70	$60 < 70 \rightarrow$ go left to 60	search
3	60 vs 60	found; left = NULL, right = 65 $\rightarrow$ one child	
4	splice	return 65, so 70.left $\leftarrow$ 65	D2

Rule D2 (one child). The search finds 60 as the left child of 70. 60 has only a right child, 65. We remove 60 by giving its parent the child directly: 70's left pointer now points to 65. The BST property still holds because 65 was already in 70's left subtree and in 50's right subtree, so $50 < 65 < 70$. No other node moves.

(b) Pseudocode

Algorithm 10 DELETE (Case 3 fragment: only a right child)

Input: node = 60, reached by DELETE searching from the root

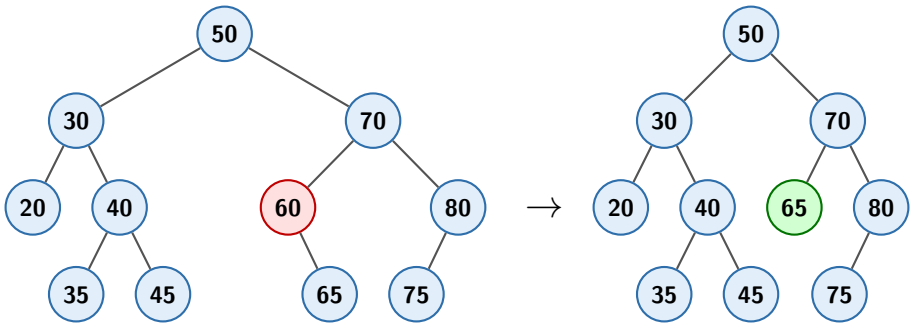
Output: 65, which becomes the new left child of 70

Initial call: root $\leftarrow$ DELETE(root, 60)

```
1. if node.left = NULL then                ▷ D2: only a right child
2.     temp ← node.right                   ▷ save the child (65)
3.     free node                           ▷ remove 60
4.     return temp                         ▷ parent's pointer now holds 65
5. end if
```

Explanation: DELETE was called as $70.left \leftarrow$ DELETE(70.left, 60), so the value returned on line 4 is stored straight into 70's left pointer.

(c) Tree Before and After



Before: 60 (red) has only a right child. After: 65 (green) takes its place as the left child of 70.

Case 4.
Delete a node with only a left child (delete 80)

(a) Rule(s) and Explanation

Step	Check	Result	Rule
1	80 vs 50	$80 > 50 \rightarrow$ go right to 70	search
2	80 vs 70	$80 > 70 \rightarrow$ go right to 80	search
3	80 vs 80	found; left = 75, right = NULL $\rightarrow$ one child	
4	splice	return 75, so 70.right $\leftarrow$ 75	D2

Rule D2 (one child, mirror case). 80 has only a left child, 75, so 75 moves up to take 80's place as the right child of 70. 75 was already in 70's right subtree, so $70 < 75$ still holds and the BST property is kept.

(b) Pseudocode

Algorithm 11 DELETE (Case 4 fragment: only a left child)

Input: node = 80, reached by DELETE searching from the root

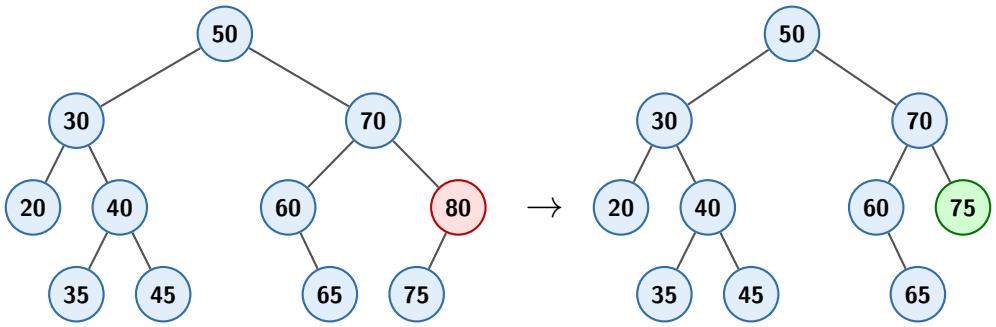
Output: 75, which becomes the new right child of 70

Initial call: root $\leftarrow$ DELETE(root, 80)

```
1. if node.right = NULL then                ▷ D2: only a left child
2.     temp ← node.left                     ▷ save the child (75)
3.     free node                            ▷ remove 80
4.     return temp                          ▷ parent's pointer now holds 75
5. end if
```

Explanation: This is the mirror image of Case 3. The search reached 80 through the call 70.*right* $\leftarrow$ DELETE(70.right, 80), so the value 75 returned on line 4 is stored straight into 70's right pointer.

(c) Tree Before and After



Before: 80 (red) has only a left child. After: 75 (green) is the right child of 70.

Case 5.
Delete a node with two children, whose successor is its right child (delete 40)

(a) Rule(s) and Explanation

Step	Check	Result	Rule
1	40 vs 50	$40 < 50 \rightarrow$ go left to 30	search
2	40 vs 30	$40 > 30 \rightarrow$ go right to 40	search
3	40 vs 40	found; children 35 and 45 $\rightarrow$ two children	D3
4	successor	right once to 45; 45.left = NULL, stop $\rightarrow$ successor = 45	D3
5	copy	the node that held 40 now holds 45	D3
6	remove old 45	old 45 has no children $\rightarrow$ remove it	D1

Rules D3, then D1. 40 has two children, so it cannot simply be spliced out. Its inorder successor is the smallest key in its right subtree. Going right once lands on 45, and 45 has no left child, so nothing in that subtree is smaller than 45: **the successor is 40's own right child** because the right subtree of 40 is just the single node 45. Copying 45 into 40's node keeps the order, since $35 < 45$ and $30 < 45 < 50$. The old 45 is then a leaf, so it is removed with D1.

(b) Pseudocode

Algorithm 12 DELETE (Case 5 fragment: two children)

Input: node = 40, reached by DELETE searching from the root

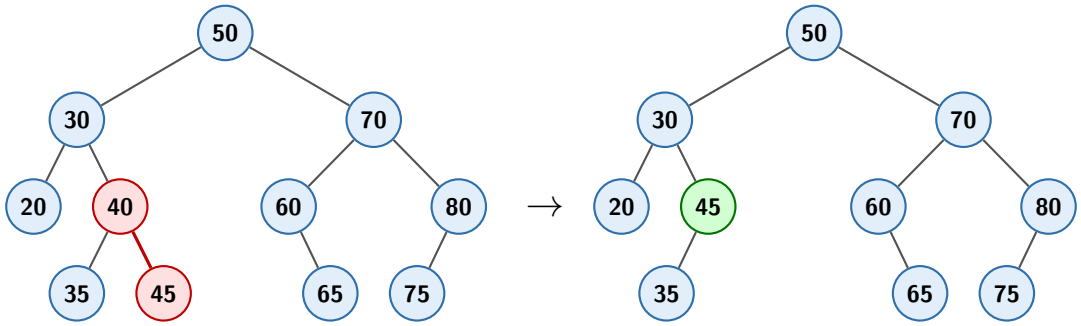
Output: the same node, now holding 45, with no right child

Initial call: root $\leftarrow$ DELETE(root, 40)

1. **if** $node.left \neq$ NULL **and** $node.right \neq$ NULL **then** ▷ D3
2. $succ \leftarrow$ FIND-MIN($node.right$) ▷ 45
3. $node.key \leftarrow succ.key$ ▷ copy 45 into 40's node
4. $node.right \leftarrow$ DELETE($node.right, succ.key$) ▷ old 45 (D1)
5. **end if**

Explanation: FIND-MIN on line 2 starts at 45 and stops immediately, because 45.left is NULL. Line 4 calls DELETE on the right subtree, which finds the old 45 as a leaf and removes it with D1, so the node now holding 45 ends up with no right child.

(c) Tree Before and After



Before: 40 and its successor 45 (red). After: 45 (green) has replaced 40, and the old 45 is gone.

Case 6.
Delete the root, whose successor has a child of its own (delete 50)

(a) Rule(s) and Explanation

Step	Check	Result	Rule
1	50 vs 50	found at the root; children 30 and 70 → two children	D3
2	successor	right once to 70, then left to 60; 60.left = NULL → successor = 60	D3
3	copy	the root now holds 60	D3
4	60 vs 70	delete old 60 from the right subtree: 60 < 70 → left to 60	search
5	60 vs 60	found; left = NULL, right = 65 → one child, 70.left ← 65	D2

Rules D3, then D2. The root has two children, so D3 applies. Its successor, 60, is the smallest key larger than 50, so it is the only key that can sit at the root with all of {20, 30, 35, 40, 45} on its left and {65, 70, 75, 80} on its right. A successor never has a left child (that child would be smaller and would be the successor instead), but it may have a right child. Here 60 has the right child 65, so deleting the old 60 uses **D2**, exactly as in Case 3.

(b) Pseudocode

Algorithm 13 DELETE (Case 6 fragment: two children, finding the successor by hand)

Input: node = 50, the root

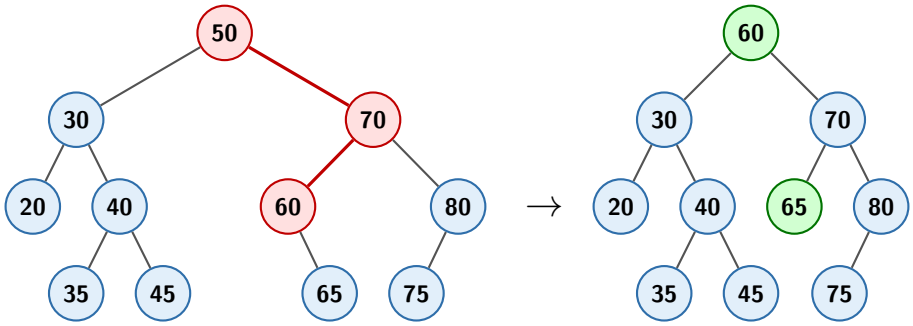
Output: the root, now holding 60, with 65 as the left child of 70

Initial call: root ← DELETE(root, 50)

```
1. if node.left ≠ NULL and node.right ≠ NULL then          ▷ D3
2.     succ ← node.right                                   ▷ go right once (70)
3.     while succ.left ≠ NULL do                           ▷ then left as far as possible
4.         succ ← succ.left                                ▷ → 60
5.     end while
6.     node.key ← succ.key                                 ▷ root now holds 60
7.     node.right ← DELETE(node.right, succ.key)           ▷ old 60: D2
8. end if
```

Explanation: Lines 2 to 5 are FIND-MIN written out: one step right, then left until the next left pointer is NULL. Line 7 deletes the old 60 from the right subtree. That call reaches 60, sees it has only a right child and applies D2. The call is made as root ← DELETE(root, 50), so the root pointer stays correct.

(c) Tree Before and After



Before: the root 50 and the path to its successor 60 (red). After: 60 is the root, and 65 has moved up under 70 (green).

Summary Table

Case	Delete	Children of the node	Successor	Rules applied	Level order after
1	33	not found	-	S1	50 30 70 20 40 60 80 35 45 65 75
2	20	none	-	D1	50 30 70 40 60 80 35 45 65 75
3	60	right only (65)	-	D2	50 30 70 20 40 65 80 35 45 75
4	80	left only (75)	-	D2	50 30 70 20 40 60 75 35 45 65
5	40	two (35, 45)	45	D3, then D1	50 30 70 20 45 60 80 35 65 75
6	50	two (30, 70)	60	D3, then D2	60 30 70 20 40 65 80 35 45 75

(g) Complete DELETE(node, key)

Algorithm 14 DELETE (Cases 1 to 6 combined)

Input: node, the root of a tree or subtree, and key, the key to delete

Output: the new root of that subtree

Initial call: root ← DELETE(root, key)

```
1.  procedure DELETE(node, key)
2.      if node = NULL then                                  ▷ S1: key not in tree (Case 1)
3.          return NULL
4.      end if
5.      if key < node.key then                               ▷ search left
6.          node.left ← DELETE(node.left, key)
7.      else if key > node.key then                          ▷ search right
8.          node.right ← DELETE(node.right, key)
9.      else if node.left = NULL and node.right = NULL then  ▷ D1 (Case 2)
10.         free node
11.         return NULL
12.     else if node.left = NULL then                        ▷ D2, right child (Case 3)
13.         temp ← node.right; free node
14.         return temp
15.     else if node.right = NULL then                       ▷ D2, left child (Case 4)
16.         temp ← node.left; free node
17.         return temp
18.     else                                                 ▷ D3 (Cases 5, 6)
19.         succ ← FIND-MIN(node.right)                      ▷ inorder successor
20.         node.key ← succ.key                              ▷ copy it up
21.         node.right ← DELETE(node.right, succ.key)        ▷ D1 or D2
22.     end if
23.     return node
24. end procedure
```

Explanation: Lines 5 to 8 are the search phase: each recursive call goes one level down and stores the returned subtree back into the same pointer, which is how every splice gets linked to its parent. Once the key is found, exactly one of D1 (line 9), D2 (lines 12 and 15) or D3 (line 18) applies, depending on the number of children. The D3 call on line 21 always ends in D1 or D2, because the successor has no left child.

(h) Why Copy the Inorder Successor Instead of Moving the Deepest Node?

In a general binary tree there is no ordering, so any node can fill the gap, and the deepest node is just the easiest one to detach. A BST has an ordering to keep: the key that replaces the deleted one must be **larger than every key in the left subtree and smaller than every key in the right subtree**. Only the inorder successor (or the inorder predecessor) is guaranteed to satisfy that, because it is the next key in sorted order.

The deepest node usually breaks the BST property. For example, moving the deepest node 75 into the root would leave 60, 65 and 70 in the root's right subtree even though they are smaller than 75.

The successor is also cheap to remove: it never has a left child, so removing it is always the easy D1 or D2 case and never another D3. The whole delete stays $\Theta(h)$.